

Automated Review System

1. Project Overview

An automated review system analyzes customer reviews and predicts the corresponding star ratings. Natural language processing (NLP) is used to extract sentiment and key features from the review texts.

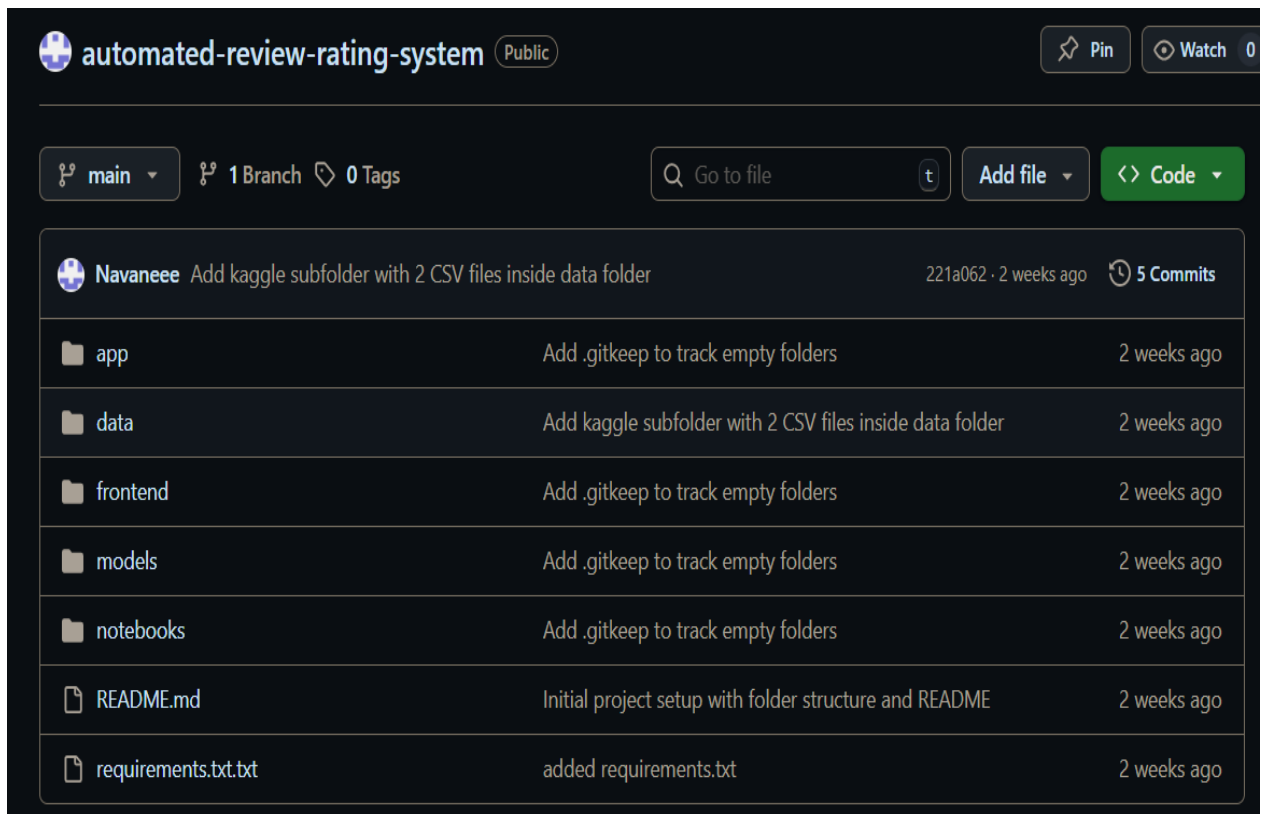
2. Environment Setup

Installed Python 3.13.3 and the required libraries such as pandas, numpy, matplotlib, seaborn, and scikit-learn. VS Code was used as the IDE. For data preprocessing, NLP, and visualization, these packages were utilized.

3. GitHub Project Setup

Created a GitHub repository named automated-review-rating-system.

Structure of directory.



The screenshot shows the GitHub interface for a public repository named 'automated-review-rating-system'. The repository is owned by 'Navaneee' and has 5 commits. The commit history table lists the following files and folders:

File/Folder	Commit Message	Commit Hash	Time
app	Add .gitkeep to track empty folders	221a062	2 weeks ago
data	Add kaggle subfolder with 2 CSV files inside data folder	221a062	2 weeks ago
frontend	Add .gitkeep to track empty folders	221a062	2 weeks ago
models	Add .gitkeep to track empty folders	221a062	2 weeks ago
notebooks	Add .gitkeep to track empty folders	221a062	2 weeks ago
README.md	Initial project setup with folder structure and README	221a062	2 weeks ago
requirements.txt.txt	added requirements.txt	221a062	2 weeks ago

4. Data Collection

The dataset was collected from Kaggle and it consists of 568454 Amazon product ids, customer reviews, star ratings and 7 more features also.

Dataset link : https://github.com/Navaneee/automated-review-rating-system/tree/main/data/kaggle/review_data.csv

5. Data Preprocessing.

5.1 Data Preview and Dropping Unnecessary Columns

The Amazon review data consisted of 10 different columns. It preprocessed to include only review ratings and review text. All other columns were removed. The preview is given below. (568454, 2) is the dimension of the new data.

```
data.head()
```

	rating	review_text
0	5	I have bought several of the Vitality canned d...
1	1	Product arrived labeled as Jumbo Salted Peanut...
2	4	This is a confection that has been around a fe...
3	2	If you are looking for the secret ingredient i...
4	5	Great taffy at a great price. There was a wid...

5.2 Missing Values

There were no missing values present in the data.

5.3 Duplicate Values Handling

There are 174779 number of duplicate rows were present in the data and all of them were removed with the code

```
data.drop_duplicates(inplace=True)
```

Also 96 duplicate reviews were present in review text column. Those were also removed with the code.

```
data.drop_duplicates(subset=['review_text'], inplace=True)
```

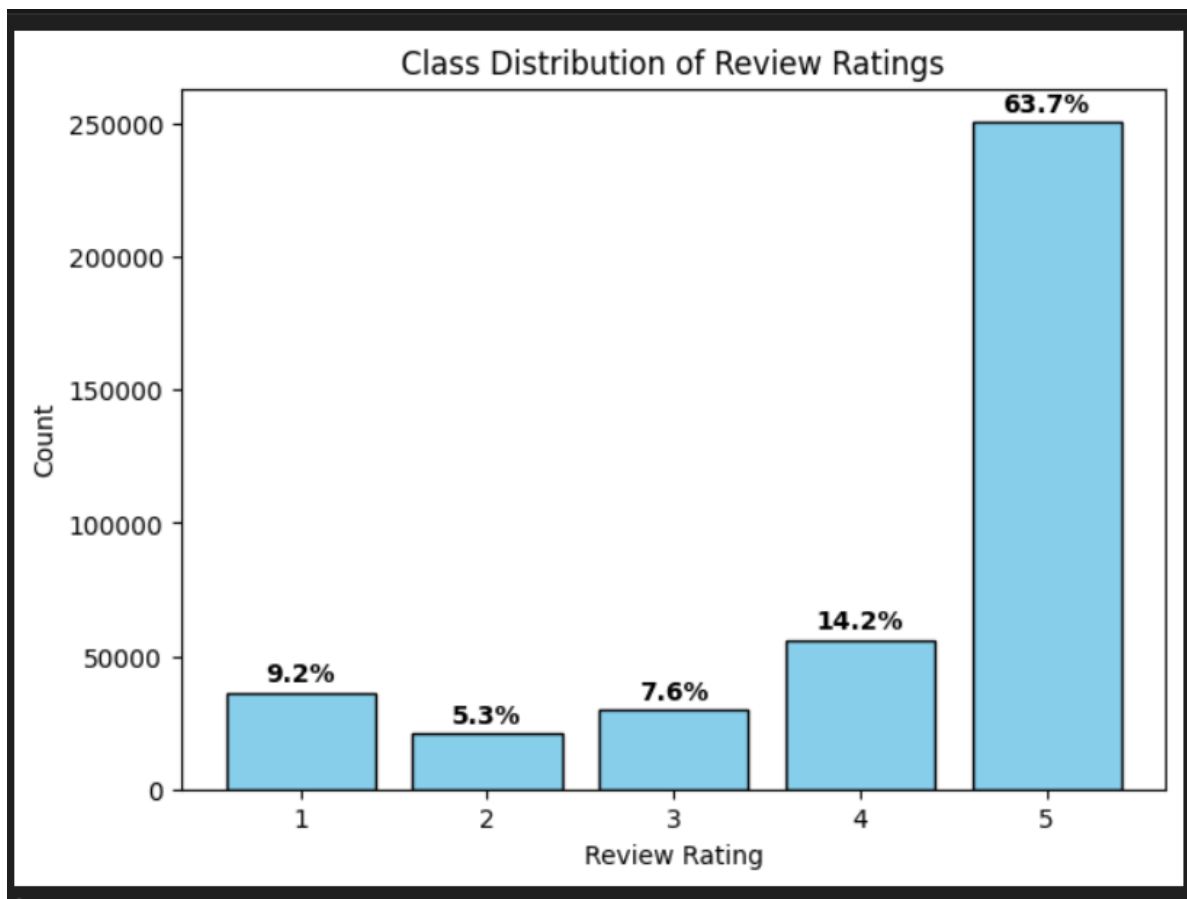
The final shape of the data was (393579, 2)

5.4 Class Distribution

The total value count of each rating is found with using `value_count()` method and it shown as

```
Class Distribution (Counts):  
rating  
1      36275  
2      20792  
3      29754  
4      56042  
5     250716  
Name: count, dtype: int64
```

The distribution of ratings is then plotted with a barplot.



The inferences from the barplot were;

- 1.The dataset is heavily imbalanced.
- 2.5-star reviews dominate and it's around 64%
- 3.2-star reviews are very low, around 5%

4. To make the data balance, we try random downsampling to the high ratings, 250,716 ratings are reduced to 50,000 and all other ratings are increased to 50,000.

5.5 Splitting the dataset to Balanced data and Imbalanced data.

The whole data is split into a balanced data with the code

- `#define the sample size(half of class 2 star ratings' count)`
`sample_size=class_counts[2]//2`
- `#Sample from rating 2`
`#Selecting all the rows where rating column =1 and sample size is half of lowest count class`
`df2_list=data[data['rating']==2].sample(n=sample_size,random_state=42) #ensures the sampling is reproducible`
- `#Sample from rating 1,3,4,5`
`#selecting same number of sample size rows from all other ratings' class`
`dfr_list=[data[data['rating']==r].sample(n=sample_size,random_state=42) for r in [1,3,4,5]]`
- `#concatenate df2_list with all others( df2,[df1,df3,df4,df5])`
`balanced_dataset = pd.concat([df2_list] + dfr_list)`
- `#Create second dataset consisting of remaining rows`
`second_dataset = data.drop(balanced_dataset.index)`
- `#Shuffle the combined dataset`
`balanced_dataset = balanced_dataset.sample(frac=1, random_state=42).reset_index(drop=True)`

and to an imbalanced dataset with the code

#setting the target percentages

- `percentage_dict={1:0.10, 2:0.15, 3:0.25, 4:0.30, 5:0.20}`

#total number of rows in the dataset

- `total=len(second_dataset)`

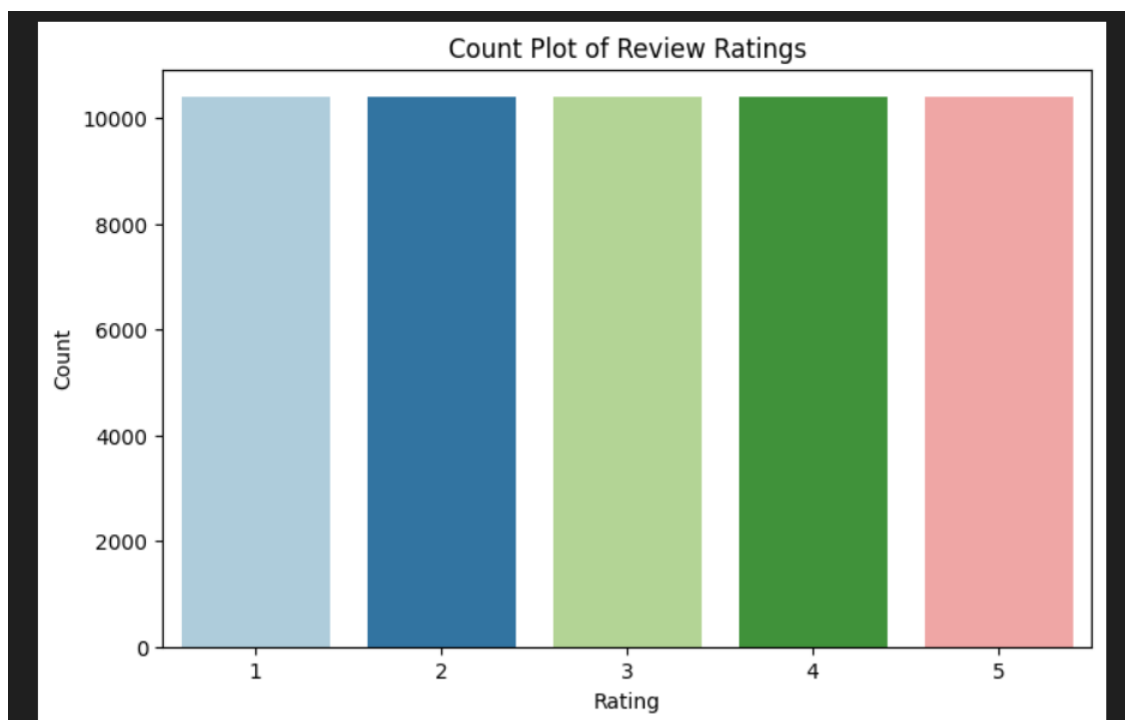
create an empty list to keep new data

- `new=[]`

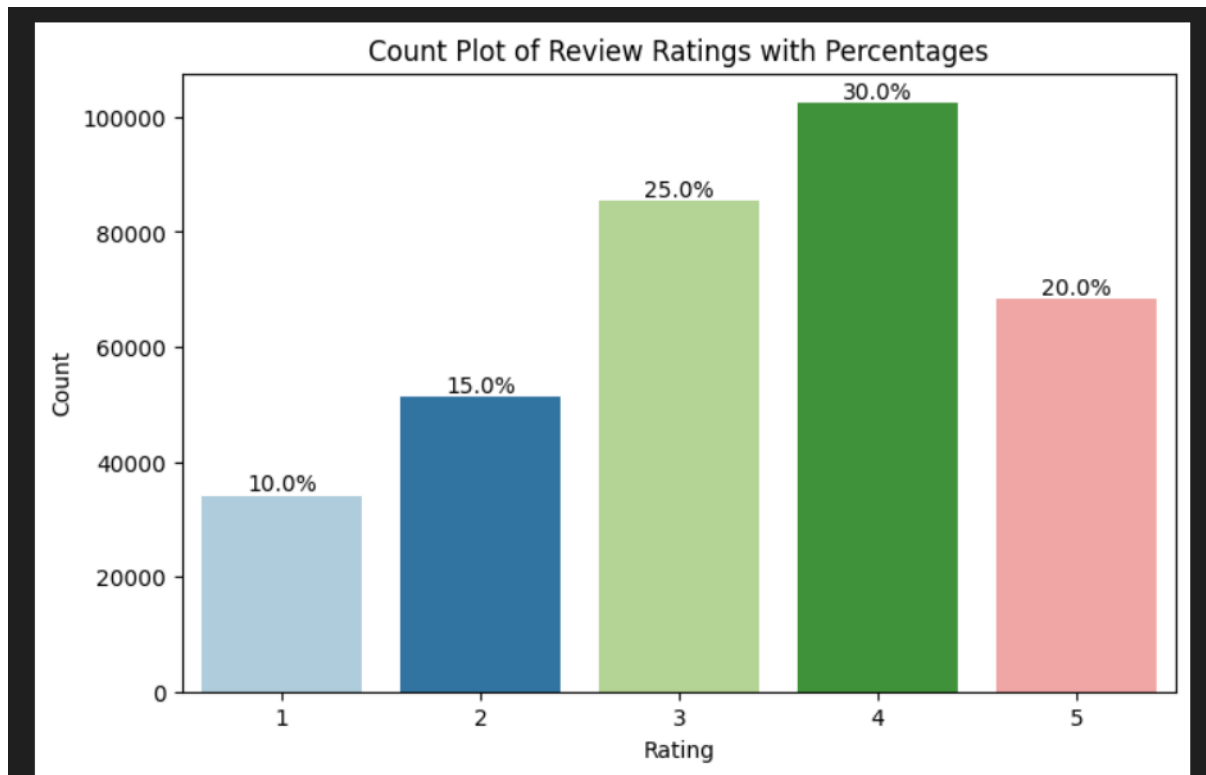
#for each ratings,we take number of rows we want according to the target percentage

- `for ratings,percentage in percentage_dict.items():`
 `total_rows_needed=int(percentage*total) #how many total rows we want to get desired percentage`
 `rows=second_dataset[second_dataset['rating']==ratings] #select only that rating(filtering the rows)`
 `sample_rows=rows.sample(total_rows_needed,replace=True,random_state=42)`
 `#select the needed rows from the data,`
 `#if not enough, repeat some rows by replace=True`
 `new.append(sample_rows)`

The countplot of balanced data



The countplot of imbalanced data



6. Balanced Dataset

6.1 Importing all the necessary libraries

- `import string` #Contains string constants and utility functions and used to remove punctuation from text
- `import nltk` #natural language processing(NLP).Provides tools for tokenization, stemming, lemmatization, POS tagging, etc
- `from nltk.corpus import stopwords` #Imports a list of common stopwords("the", "is", "and", ...) from NLTK and helps in removing stopwords from text to reduce noise in NLP tasks.
- `from nltk.stem import WordNetLemmatizer` #reducing words to their base form.
- `from sklearn.model_selection import train_test_split` #split datasets into training and testing sets

- from sklearn.feature_extraction.text import TfidfVectorizer #converts text into TF-IDF features(num vectors representing text importance) ,transforms text data into a format suitable for machine learning models

6.2 Text Preprocessing

Created a user defined function caeled “preprocess_text”. This function cleans and prepares the review text for analysis using Natural Language Processing (NLP) techniques.

1. Lowercasing:

Converts all text to lowercase to maintain consistency.

2. Removing URLs and HTML tags:

Eliminates links and HTML elements that are not useful for text analysis.

3. Removing emojis and special characters:

Keeps only alphabetic characters to focus on meaningful words.

4. Tokenization:

Splits the text into individual words (tokens).

5. Stopword Removal:

Commonly used words that do not contribute much to the meaning or sentiment of a sentence are removed.

The list of stopwords used comes from NLTK’s default English stopwords set, which includes words such as:

```
{'by', 'yourself', 've', 'been', 'other', 'in', "didn't", 'each', 'this',
'he'll', 'am', "they've", "you're", 'ourselves', 'ours', 'doing', "it'll",
'shan't', 'hadn', 'most', 'than', 'they', "we'd", 'she', 'haven', "couldn't",
'hasn't', 'his', 'out', 'over', "we'll", "isn't", 'against', 'which', 'hers',
'or', "won't", "you'd", 'above', 'should', 'on', 'because', 'he', "she'll",
'does', 'couldn', 'shan', "don't", "they'd", 'with', 'a', 'why', 'weren',
'shouldn', 'yourselves', 'ain', "she'd", 'more', 'll', 'myself', 'theirs',
'but', 'about', 'them', 'did', 'to', 'has', 'be', 'so', 'of', 'too', 'while',
'should've", 'those', 'don', "she's", 'some', 'all', 'such', "aren't", 'from',
'here', 'whom', "mightn't", 'wouldn', 'm', "you've", 'at', 'themselves',
'mustn', 'if', 'then', 'now', 'nor', 'that', "they're", 'won', 'no', 'have',
'me', 'were', 'own', "we've", 'any', 'o', 'do', 'mightn', "we're", "he's",
```

"shouldn't", 'further', 'was', 'there', 'is', 'its', 'between', 'for', 'only',
 'an', 'my', 'i', "haven't", 'where', "hadn't", 't', 'what', 'into', 'once',
 'below', 'himself', 'having', 'are', 'ma', 'off', 'we', 'who', 'before',
 'yours', 'hasn', 'not', 's', 'as', 'doesn', 'didn', 'him', "that'll", 're',
 "it'd", 'isn', 'it', 'can', 'her', 'd', "weren't", 'had', 'itself', 'their',
 'herself', "you'll", 'again', 'you', 'down', 'your', "i'd", "it's", 'through',
 'under', 'very', 'wasn', 'during', 'how', "wasn't", "mustn't", "i've", 'both',
 'until', 'y', "i'm", 'and', 'the', 'after', 'when', 'few', 'aren', 'our',
 "they'll", 'up', 'being', "i'll", 'needn', 'these', 'same', 'will',
 "wouldn't", "he'd", 'just', "doesn't", "needn't"}

6. Lemmatization:

Each word is converted to its base or dictionary form (e.g., *“running”* → *“run”*), helping the model treat similar words as the same.

7.Rejoining:

The cleaned tokens are combined back into a single string.

The cleaned text is saved in a new column *“cleaned_review”*, and a *“word_count”* column is created to store the number of words in each processed review.

8. Removed too long or short texts:

The review texts above word below 3 and above 175 is removed with the code

- `data = data[(data["word_count"] >= 3) & (data["word_count"] <= 175)]`

6.3 Data Visualization

A subplot consisting of a barplot ,histogram and box[plot of word count by ratings plotted.

#subplots of 3 rows and 1 column

- `fig, axes = plt.subplots(3, 1, figsize=(10, 15))`

#bar plot: Review count per rating

- `sns.countplot(x="rating", data=data, palette="Paired", ax=axes[0])`

- `axes[0].set_title("Review Count per Rating")`
- `axes[0].set_xlabel("Rating")`
- `axes[0].set_ylabel("Count")`

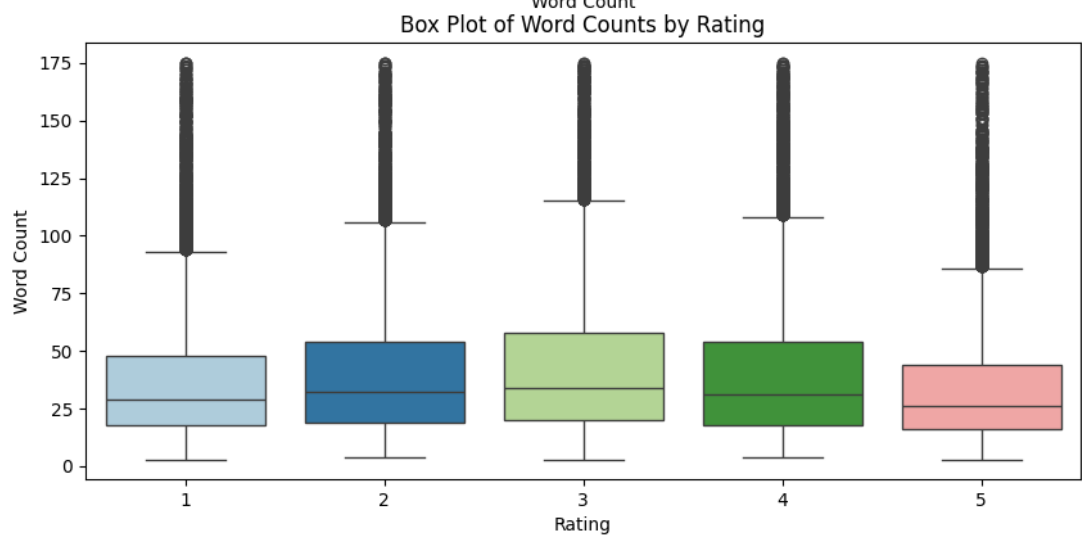
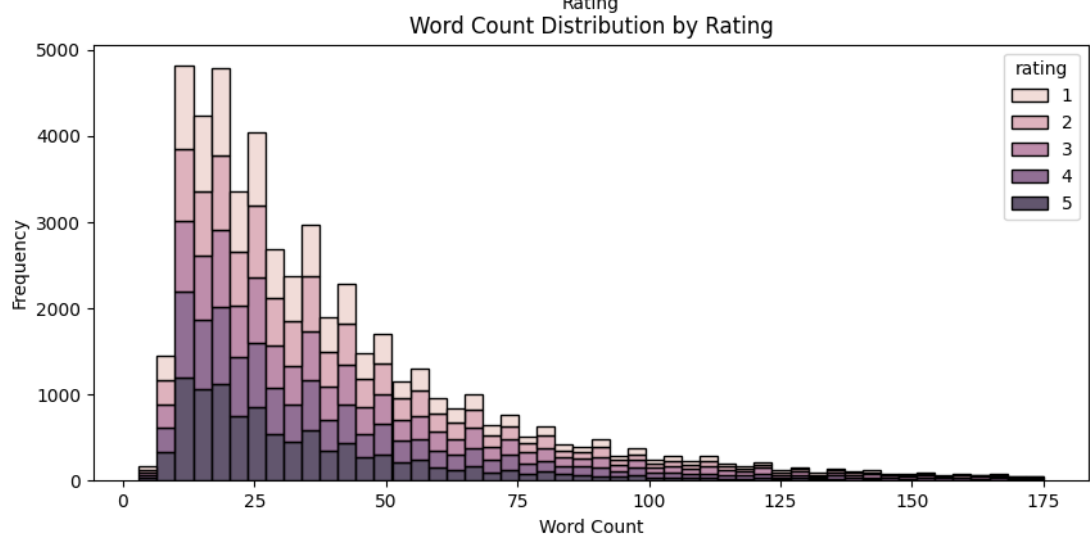
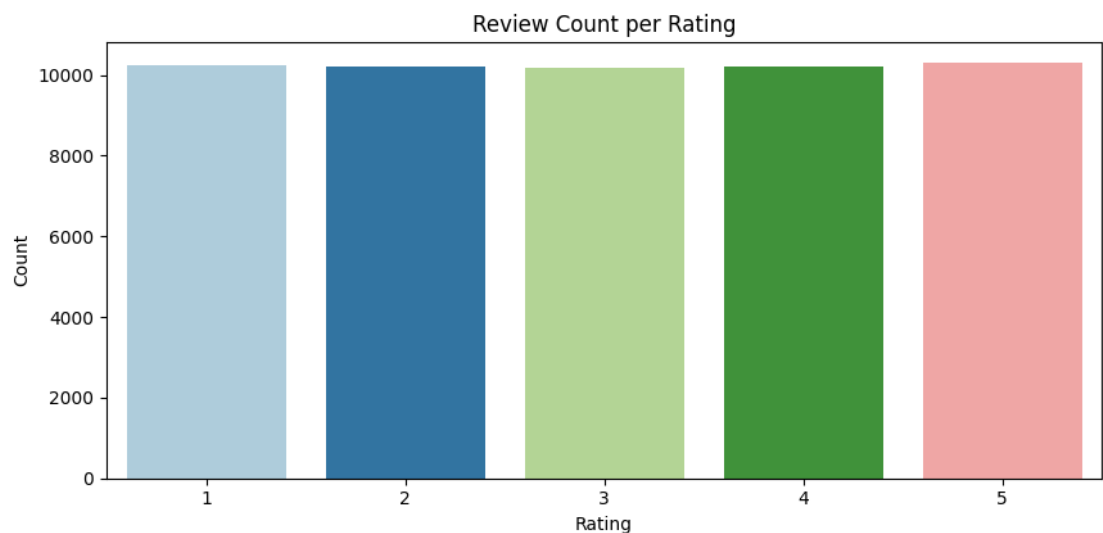
`#histogram of word counts`

- `sns.histplot(data, x="word_count", hue="rating", multiple="stack", bins=50, ax=axes[1])`
- `axes[1].set_title("Word Count Distribution by Rating")`
- `axes[1].set_xlabel("Word Count")`
- `axes[1].set_ylabel("Frequency")`

`#box plot of word counts`

- `sns.boxplot(x="rating", y="word_count", data=data, palette="Paired", ax=axes[2])`
- `axes[2].set_title("Box Plot of Word Counts by Rating")`
- `axes[2].set_xlabel("Rating")`
- `axes[2].set_ylabel("Word Count")`

- `plt.savefig("bal.png")`
- `plt.tight_layout()`
- `plt.show()`



The inferences from the plots are;

- Barplot
 - The bar heights are same, means balanced the dataset across all ratings from 1 to 5.
- Histplot
 - Most reviews have fewer words and around 50. The distribution is right-skewed, large number of short reviews, and fewer long ones.
 - The overlap between ratings shows review length is not a strong indicator of sentiment/rating. Both good and bad reviews can be short or long.
- Boxplot
 - Median word counts are around 25–35 words
 - The interquartile ranges are also similar, meaning review length doesn't vary much between 1-star and 5-star ratings.
 - Many outliers exist (long reviews above 100 words). These could be detailed opinions or complaints.
 - Slightly shorter reviews appear in extreme ratings (1 and 5), while middle ratings (2 to 4) show more spread, possibly reflecting more nuanced feedback.
 - Outliers which are long reviews may carry rich detail, potentially useful for deeper analysis of sentiment.

Sample ratings preview

sample review of rating 1

['wondering much cheaper flavor know cat wouldnt touch acted like stunk backed awayhe went eat day old dried cat food rather eat junkim concerned someone wrote might tainted recalled later well ill save case get recalledmaybe get refund dont like amazon wont let return food itemsunless recalled especially unopened cansalot people gave good rating alot people didnt think one cat like odds bad shouldnt buy', 'ordered box newman dried apricot understood dark color lack preservative ate anyway became illvomiting nausea chill dizzinessabout hour eating occur apricot thought bad virus soi ate slow learnersame result miserable contacted amazon gracious refunding moneythe remainder apricot trash warned','already ordered directly cafe britt jumped chance order cafe britt product amazon thus saving shipping cost even getting break price experience dealing cafe britt great ordered lb bag whole bean costa rica dark roast coffee loved wanting try something different ordered lb bag tres rio valdivia whole bean coffee thru amazomcom upon opening first bag noticed difference quality quite apparent assumed probably different type coffee opening second bag think looking freshness date packaging ordered coffee july received august rd package

freshness date jun lot appears amazon shipped coffee way date know local grocery store going sell food date amazon itsecond entryhaving spoken customer service rep amazon told outofdate product replaced cost appreciate must commend amazon responsiveness complaint', 'ordered product received broken bottle amazon excellent reshipped entire order one day shipping unfortunately even cent bottle overpaid cheese taste really salty mess bottle average product save money', 'natural organic granola mix wonderful unfortunately contains gmo hence cannot consumed']

sample review of rating 2

['parachute coconut oil really coconut kind hard tell coconut oil smell like sort bacon fat tried use dont like smell bacon product totally bad', 'mom turned alba milkshake remember tasting pretty good thrilled find amazon year ordered item massive quantity im trying find people give since cant force fake aftertasteladen stuff throat dont know different ingredient faulty memory stuff terribly disappointing borderline disgusting', 'dog loved taste dog food didnt agree system month old lab puppy amount food recommended much food dog think stuff expensive also', 'coffee bean purchased directly roaster typically arrive hour time roasted always ship within hour roasted inexpensive single sourced precisely roasted bean brewed correctly produce unbeatable cup coffeewhen purchased amazon unfortunately bean arrived day roasted date day roasted printed bag format dayoftheyearyear bag amazon came printed meaning st day june today august shipped day agocoffee peak moment come roaster decline every minute via dissipation volatile aromatics oxidation staling buy directly roaster want fresh buy amazon want cheap mediocre halfstale coffee bulk', 'listing say milk chocolate covered however received dark chocolate covered peanut really dont like dark chocolate much ended giving away part ate theyre alright listed']

7. Train Test Split

- The dataset is split into training (80%) and testing (20%) sets using the `train_test_split()` function.
The `stratify=y` parameter ensures that the proportion of each rating category remains the same in both the training and test sets.
- `x = balanced["cleaned_review"] # training`
- `y = balanced["rating"] # testing`
- `x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, stratify=y, random_state=42)`

8. TF-IDF Vectorization

TF-IDF is a statistical measure that evaluates how important a word is in a document relative to a collection of documents (corpus). It combines two components:

1. Term Frequency (TF): Measures how often a word appears in a document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. Inverse Document Frequency (IDF): Measures how unique or rare a word is across all documents.

$$IDF(t) = \log \frac{\text{Total number of documents}}{\text{Number of documents containing term } t}$$

3. TF-IDF Score:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

Words that occur frequently in a document but rarely across all documents get higher TF-IDF scores, highlighting their importance for classification. Common words like "the" or "is" get low scores because they appear in almost all documents.

This TF-IDF representation allows the model to focus on the most relevant words for predicting review ratings

- `vectorizer = TfidfVectorizer(max_features=5000)`
- `X_train_tfidf = vectorizer.fit_transform(X_train)`
- `X_test_tfidf = vectorizer.transform(X_test)`
- `print("TF-IDF Train shape:", X_train_tfidf.shape)`
- `print("TF-IDF Test shape:", X_test_tfidf.shape)`

`#max_features=5000` limits the number of features to the top 5000 most important words in the dataset.

`#fit_transform()` learns the vocabulary from the training data and converts it into TF-IDF features.

`#transform()` converts the test data into the same feature space without learning a new vocabulary.

the textual review data is converted into numerical features using "TF-IDF (Term Frequency-Inverse Document Frequency)". Machine learning models require numerical input, so text data must be transformed into a numerical representation.

